DEEP LEARNING FUNDAMENTALS

# Recurrent Neural Networks

Basic Understanding, Architecture, and Mechanics

# Presentation Agenda

### The Basics

Understanding sequential data and the need for RNNs.

### Mechanics

Architecture, Forward Propagation, and Math.

### Applications

Real-world examples in NLP and Time Series.

# What is Sequential Data?

- **Order Matters:** Unlike standard datasets where inputs are independent, sequential data depends on the order.
- **Time Series:** Stock prices, weather data, heart rates.
- **Natural Language:** Sentences, translation (Word A depends on Word B).
- **Audio/Video:** Sound waves, video frames.

# The Limitation of Feedforward Networks

### No Memory

Standard Feedforward Neural Networks (FNNs) process each input independently. They have no concept of what happened in the previous step.

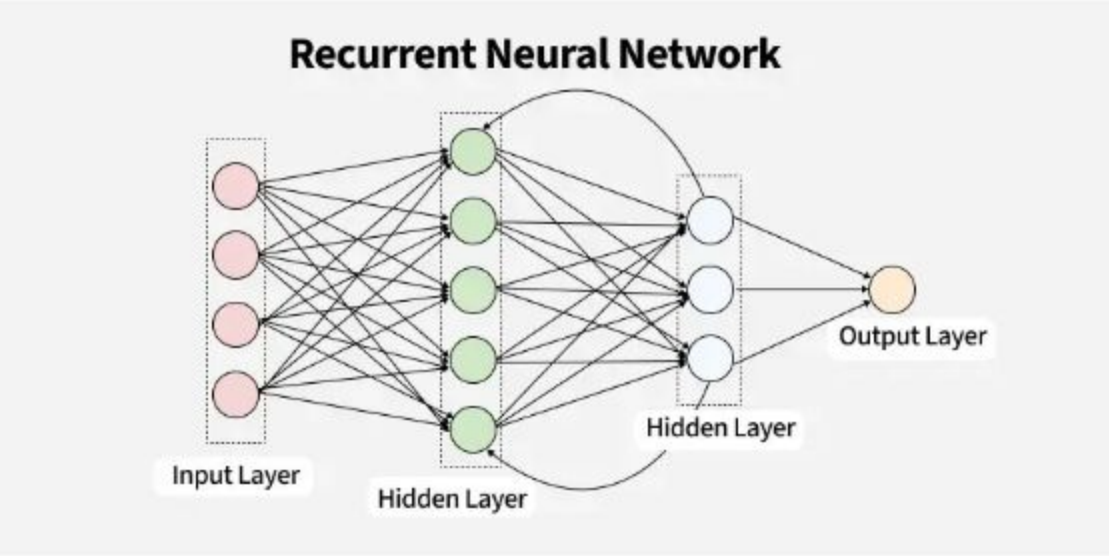
### Fixed Input Size

FNNs require a fixed-size input vector (e.g., an image of 28x28 pixels). Sequential data, like sentences, can be of variable length.

# Enter the RNN

Recurrent Neural Networks designed specifically to handle sequential data by introducing the concept of "Memory".

# The Core Idea: A Loop




### Persistence of Information

An RNN processes a sequence of vectors one by one.

While processing the current input, it passes a "hidden state" (memory) to the next step of the sequence.

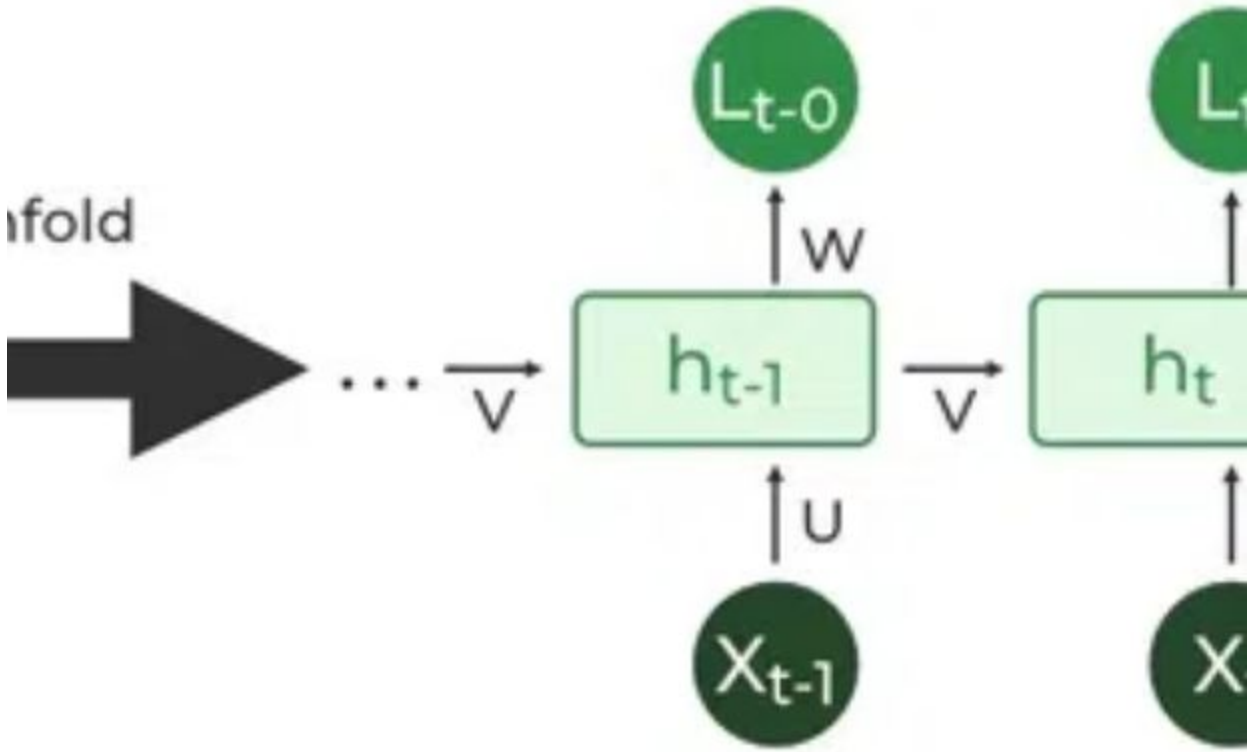
This loop allows information to persist.

## Unfolding Through Time

Although we draw RNNs as a loop, they can be visualized as a chain of repeating modules.

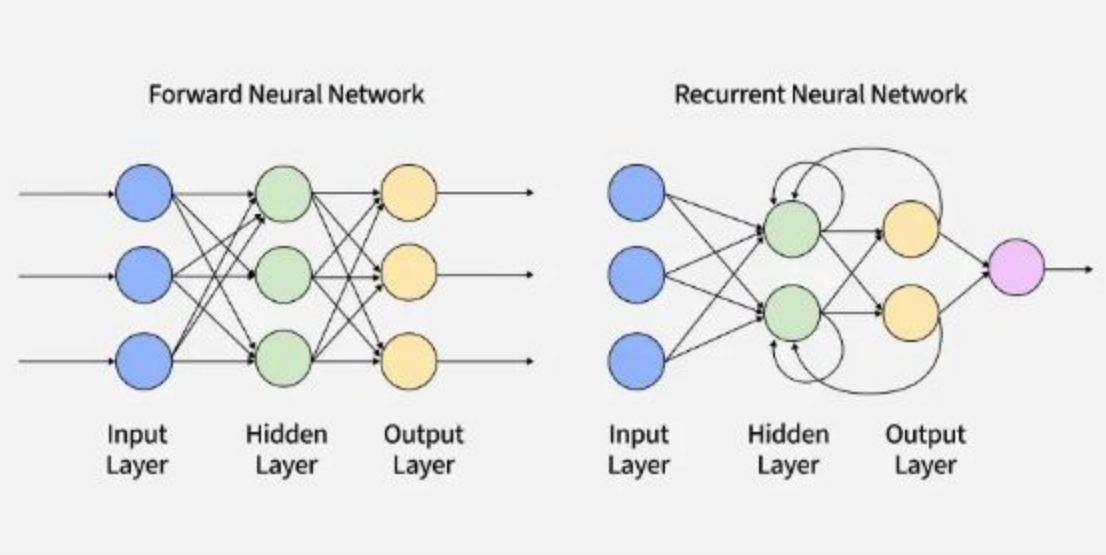
Each module corresponds to a specific time step ($t$, $t+1$, etc.) in the sequence.

This "unfolded" view reveals how the network is essentially a deep network where depth is the length of the sequence.



# FNN vs. RNN

- **Feedforward (Left):** Information flows only in one direction, from input to output. No cycles.
- **Recurrent (Right):** Includes a feedback loop where the output of a layer is added to the next input and fed back into the same layer.



# Mathematical Notation

### Variables

- $x_t$: Input vector at time step $t$.
- $h_t$: Hidden state at time step $t$ (the memory).
- $y_t$: Output at time step $t$.

### Weights (Parameters)

- $W_{xh}$: Weights connecting Input to Hidden layer.
- $W_{hh}$: Weights connecting Hidden state to Hidden state.
- $W_{hy}$: Weights connecting Hidden state to Output.

# The Concept of Shared Weights

### Parameter Sharing

Unlike traditional deep nets where each layer has unique weights, an RNN uses the **same** weight matrices ($W$) across all time steps.

### Learning Efficiency

This greatly reduces the number of parameters to learn and allows the model to apply the same knowledge to different parts of the sequence.

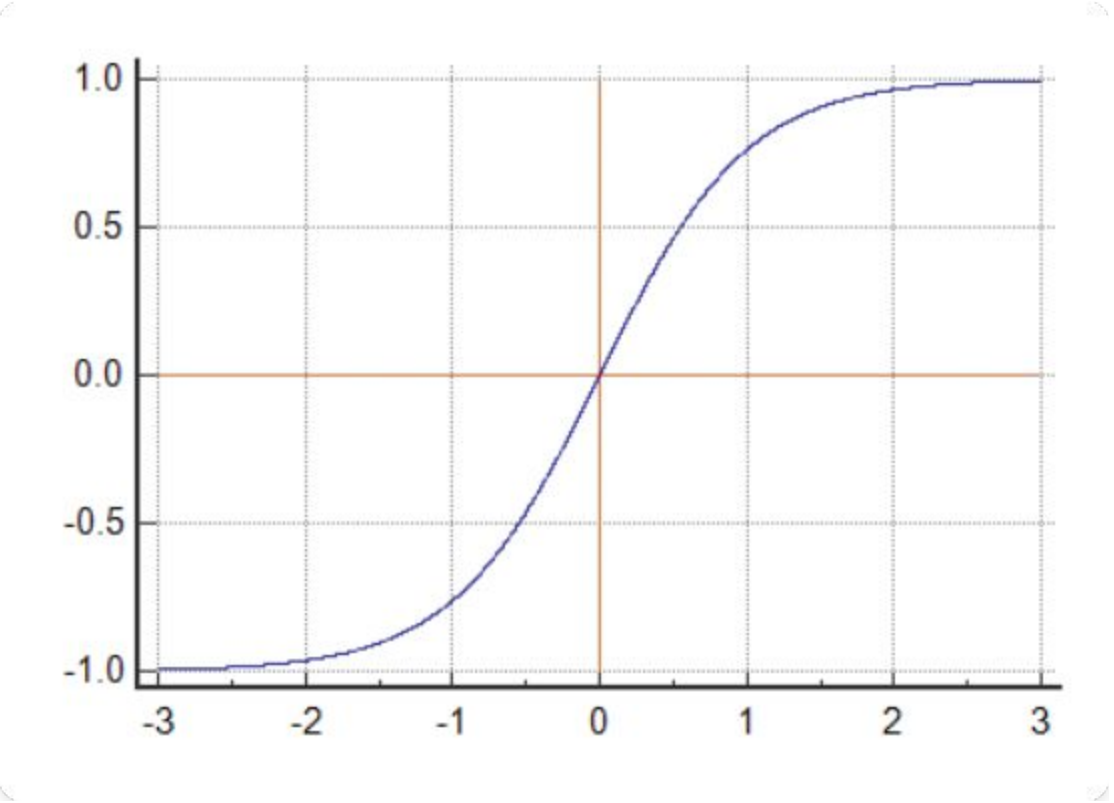
# Activation Function: Tanh

### Why Tanh?

RNNs typically use the Hyperbolic Tangent (tanh) activation function for the hidden state.

It squashes values between **–1 and 1**.

This helps regulate the flow of information and prevents values from exploding too quickly as they loop through the network.



# Step 1: Calculating Hidden State

### The Formula

$$h_t = \tanh W_{hh} h_{t-1} + W_{xh} x_t + b$$

### Explanation

The new memory ( ) is a combination of the old memory ( ) and the current input ( ). $h_t$ $h_{t-1}$ $x_t$

Both are multiplied by their respective weight matrices and passed through the tanh activation.

# Step 2: Calculating Output

### The Formula

$$y_t = W_{hy} h_t + b_y$$

### Explanation

The output at the current step ( ) is derived solely from the current hidden state ( ). $y_t$ $h_t$

Note: For classification tasks, we usually apply a Softmax function to to get probabilities. $y_t$

# Working Example: Word Prediction

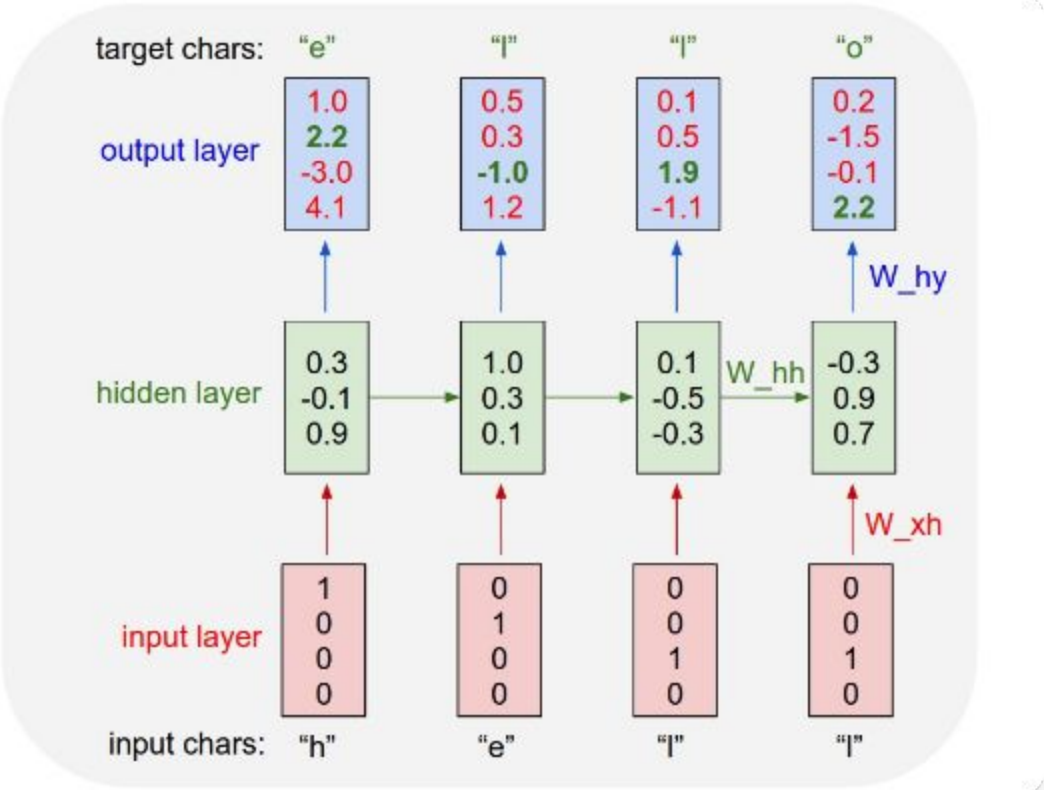
Predicting the next word in the sequence: "The cat sat on the..."

# Input: One-Hot Encoding

We convert words into numerical vectors. Vocabulary = [The, cat, sat, on, mat].

| Word | Vector ( x t ) |
| --- | --- |
| The | [1, 0, 0, 0, 0] |
| cat | [0, 1, 0, 0, 0] |

# Processing "The" (Time $t = 1$)




### Initial State

At $t = 0$, the hidden state is typically initialized to a vector of zeros, $h_0$.

### Computation

The network takes "The" ($x_1$) and $h_0$ to calculate $h_1$.

This $h_1$ now holds the "memory" of the word "The".

# Processing "Cat" (Time $t = 2$)

### Updating Memory

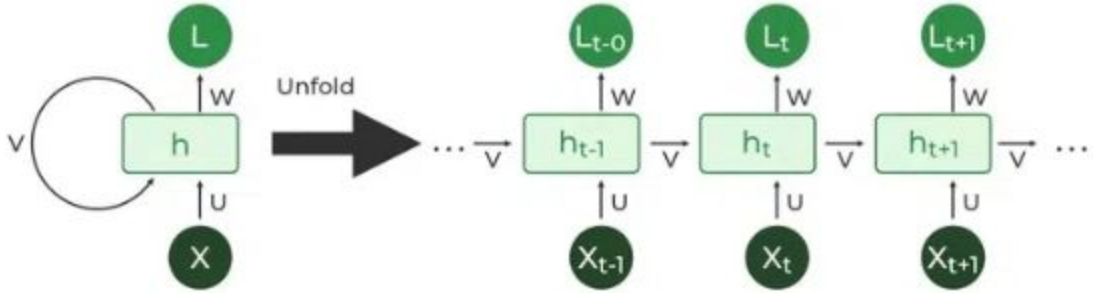
Input: "cat" ( $x_2$ )

Previous Memory: ( $h_1$ contains info about "The").

### Result

New Memory $h_2$ contains information about "The cat".

The context is growing.



# Final Prediction

"Mat"

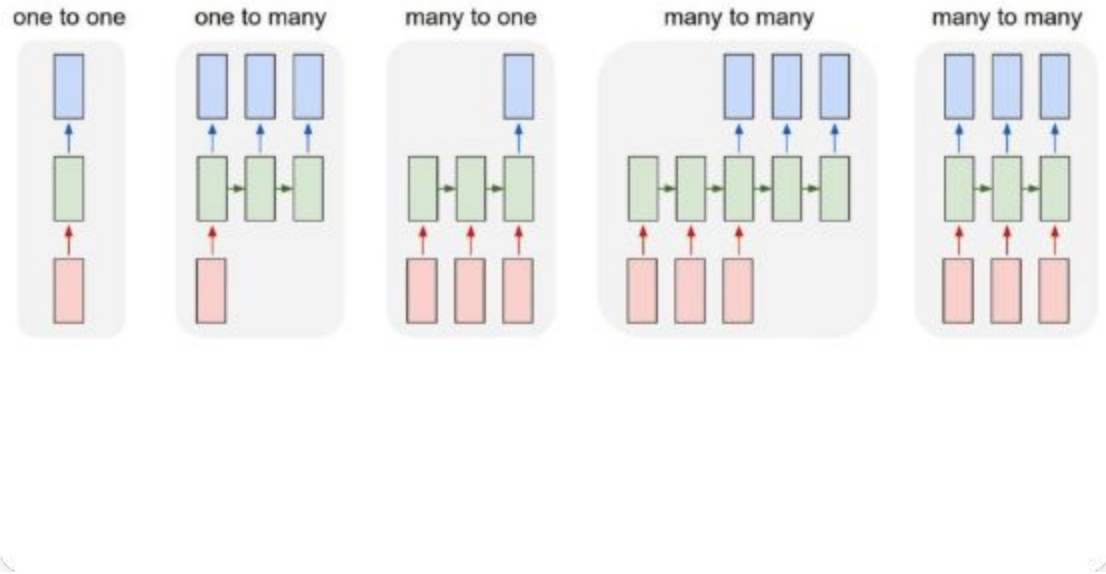
Predicted Word

### Softmax Probability

After processing "The cat sat on the", the final hidden state $h_5$ is used to calculate $y_5$.

The Softmax function assigns the highest probability to the word "mat" based on the training data context.

# RNN Type: One-to-Many




### Structure

Single Input Sequence Output.

### Application: Image Captioning

**Input:** An image (feature vector from a CNN).

**Output:** A sentence describing the image ("A dog playing in the park").

# RNN Type: Many-to-One

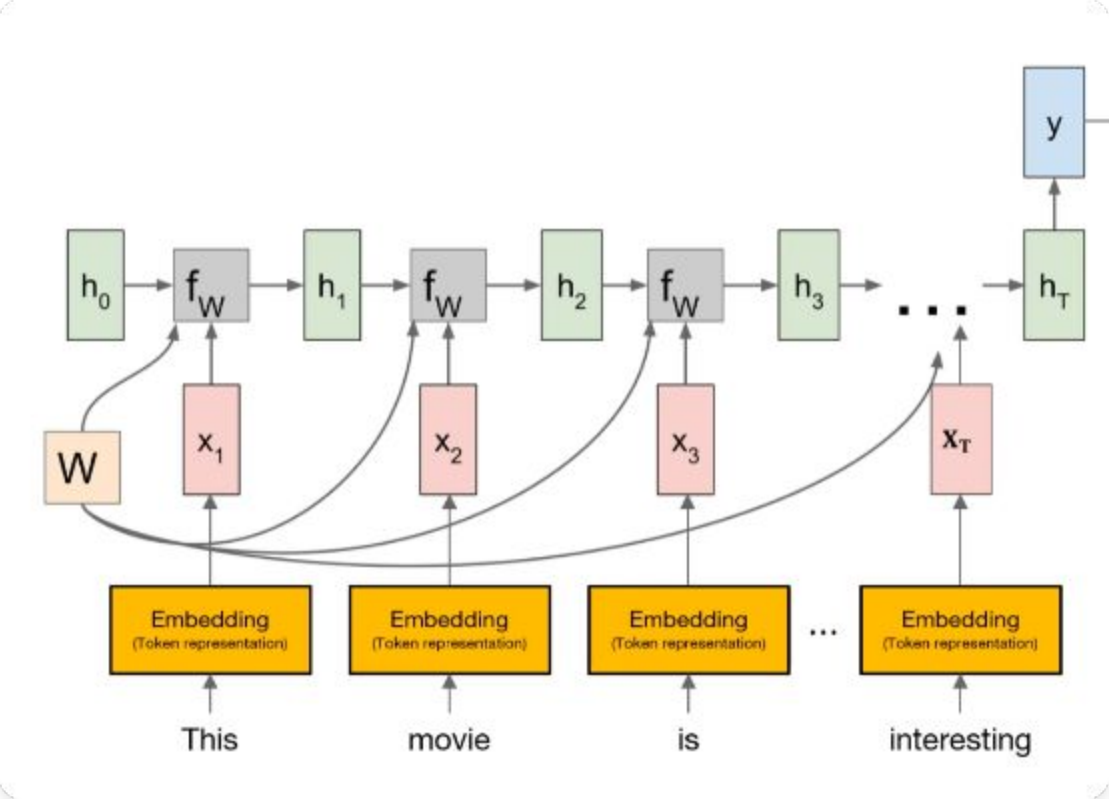
### Structure

Sequence Input Single Output.

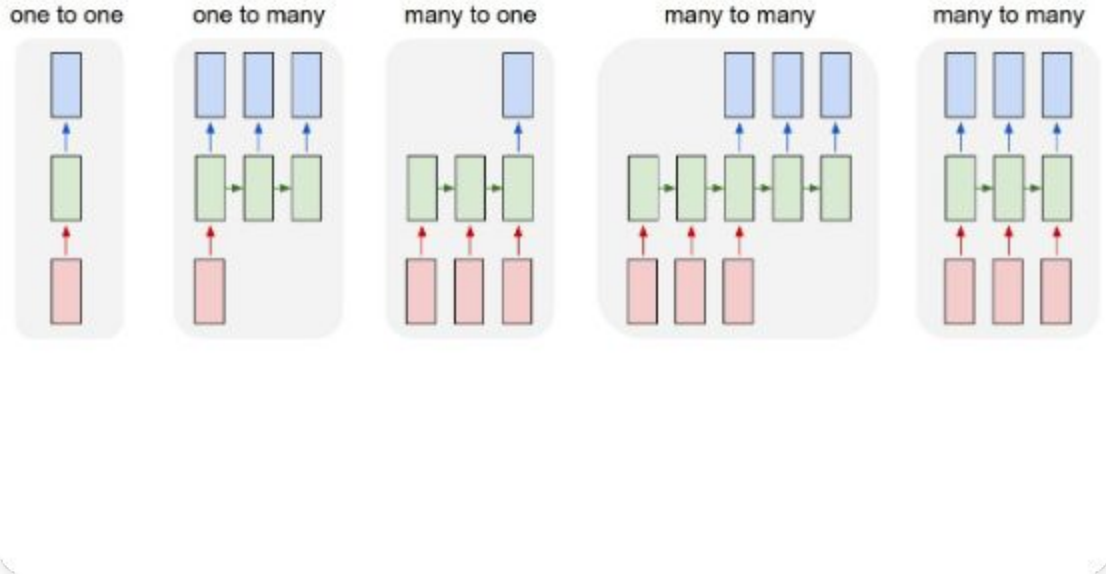
### Application: Sentiment Analysis

**Input:** A movie review (sequence of words).

**Output:** Sentiment score (Positive/Negative).



# RNN Type: Many-to-Many




### Structure

Sequence Input Sequence Output.

### Application: Machine Translation

**Input:** Sentence in English.

**Output:** Sentence in French.

This is typically an "Encoder-Decoder" architecture.

# Training: Loss Over Time

### Total Loss

$$L = \sum_{t} L_t$$

### Backpropagation

The network makes predictions at every time step.

We calculate the loss (error) at each step and sum them up to get the total loss for the sequence.

We need to update weights based on this cumulative error.

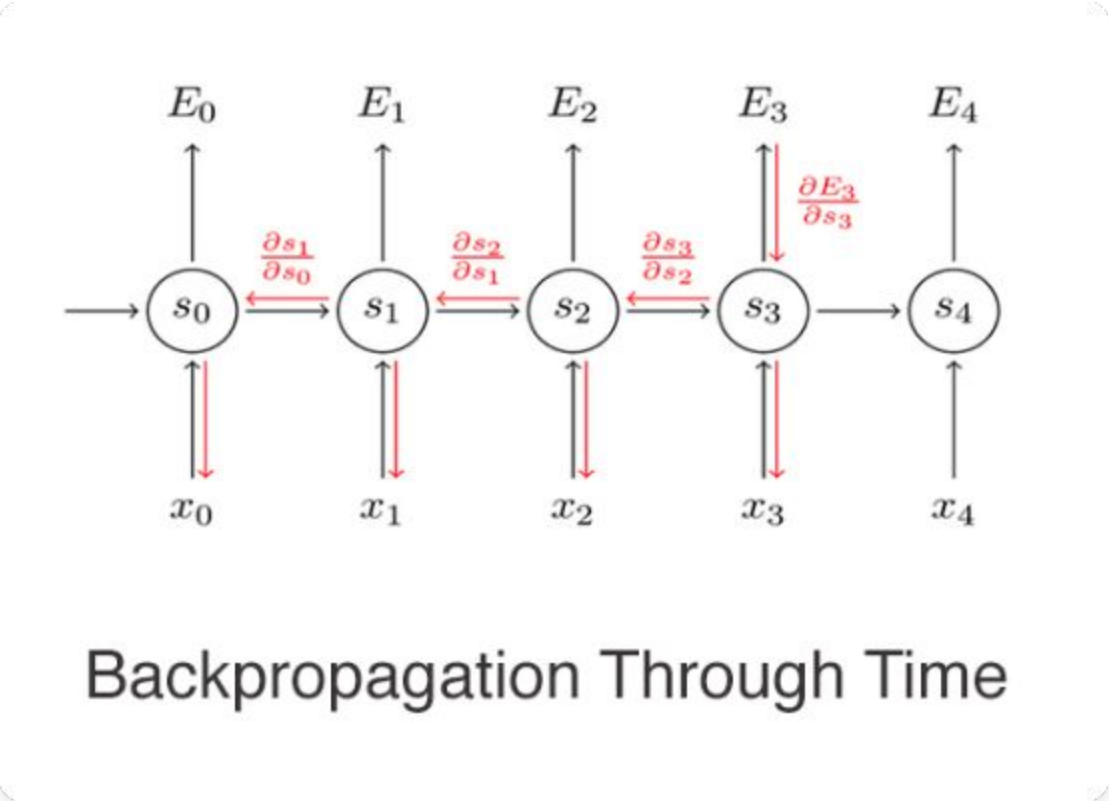
# Backpropagation Through Time (BPTT)

### The Concept

BPTT is a specific training algorithm for RNNs.

Because the parameters ( ) are shared across all time steps, the gradient at time depends on the gradient at time .

We unroll the network and propagate error backward from the last time step to the first.



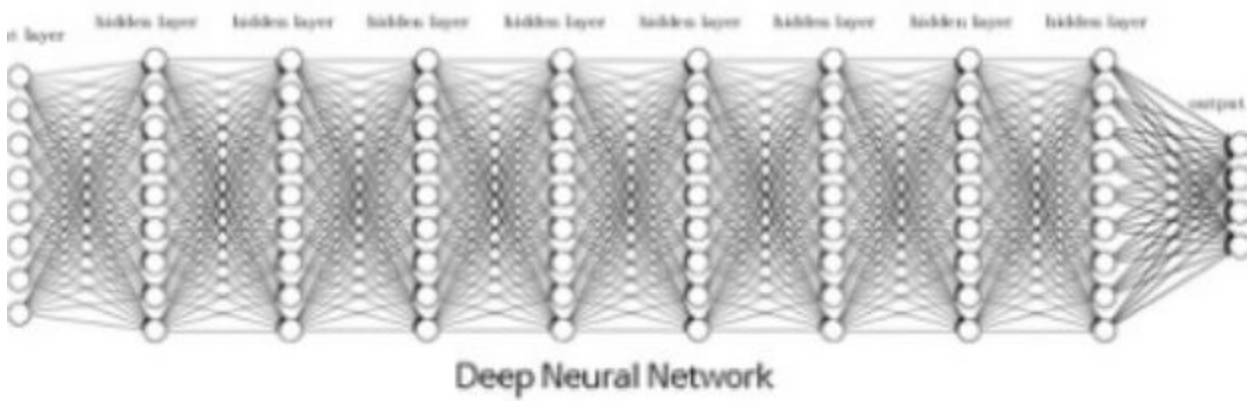
## The Problem: Vanishing Gradient

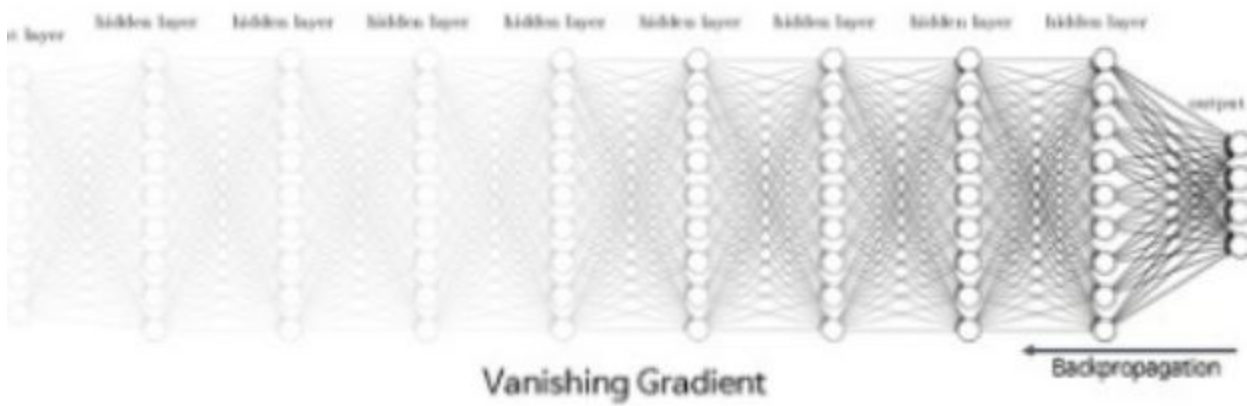
When training RNNs on long sequences, gradients are multiplied repeatedly (chain rule).

If the weights are small (< 1), the gradient shrinks exponentially as it moves back in time.

**Result:** The model forgets early inputs. It cannot learn "Long-Term Dependencies".

# l: Vanishing Gradient P

# The Opposite: Exploding Gradient

### The Mechanism

If weights are large (> 1), the gradients can grow exponentially during backpropagation.

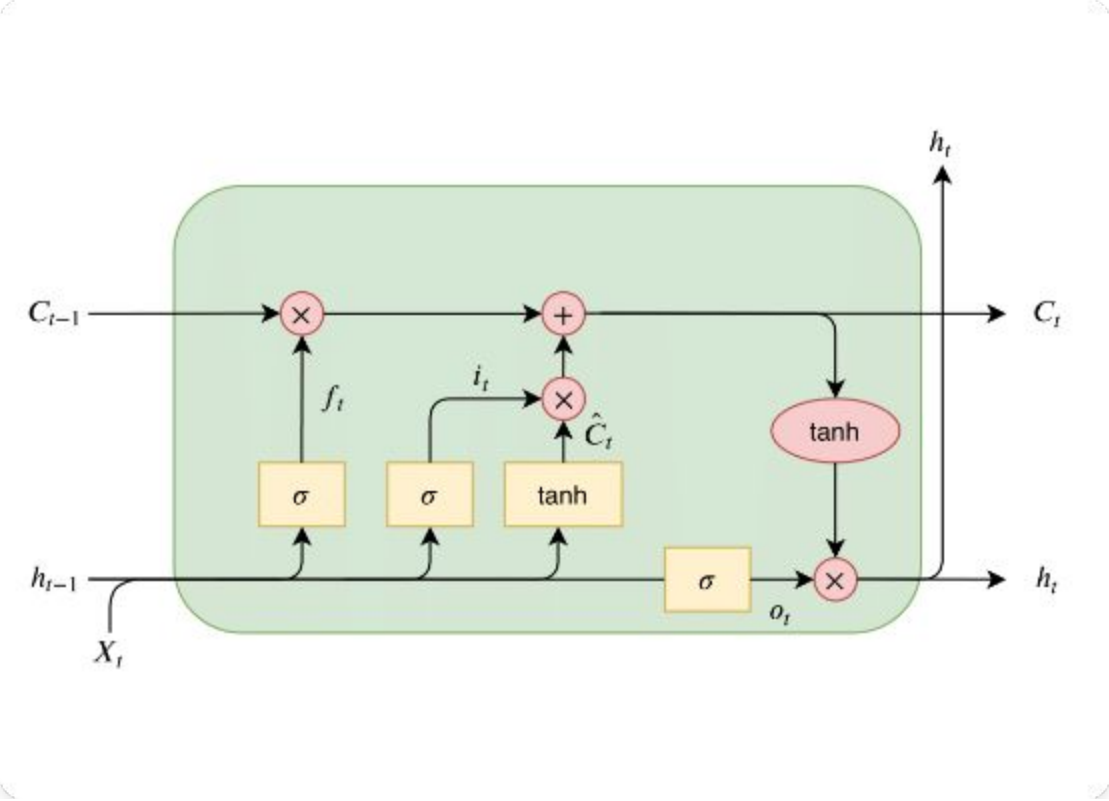
This causes the model weights to update drastically, leading to instability (NaN values).

### The Solution: Clipping

**Gradient Clipping:** A technique where we simply cap the gradients at a maximum threshold.

If gradient > threshold, set gradient = threshold.

# Solution: Long Short-Term Memory (LSTM)




### A Smarter Cell

LSTMs are a special kind of RNN capable of learning long-term dependencies.

They introduce a **Cell State** (a highway for information) and three **Gates**:

- **Forget Gate:** What to throw away.
- **Input Gate:** What to store.
- **Output Gate:** What to output.

# Alternative: Gated Recurrent Unit (GRU)

### Simplified LSTM

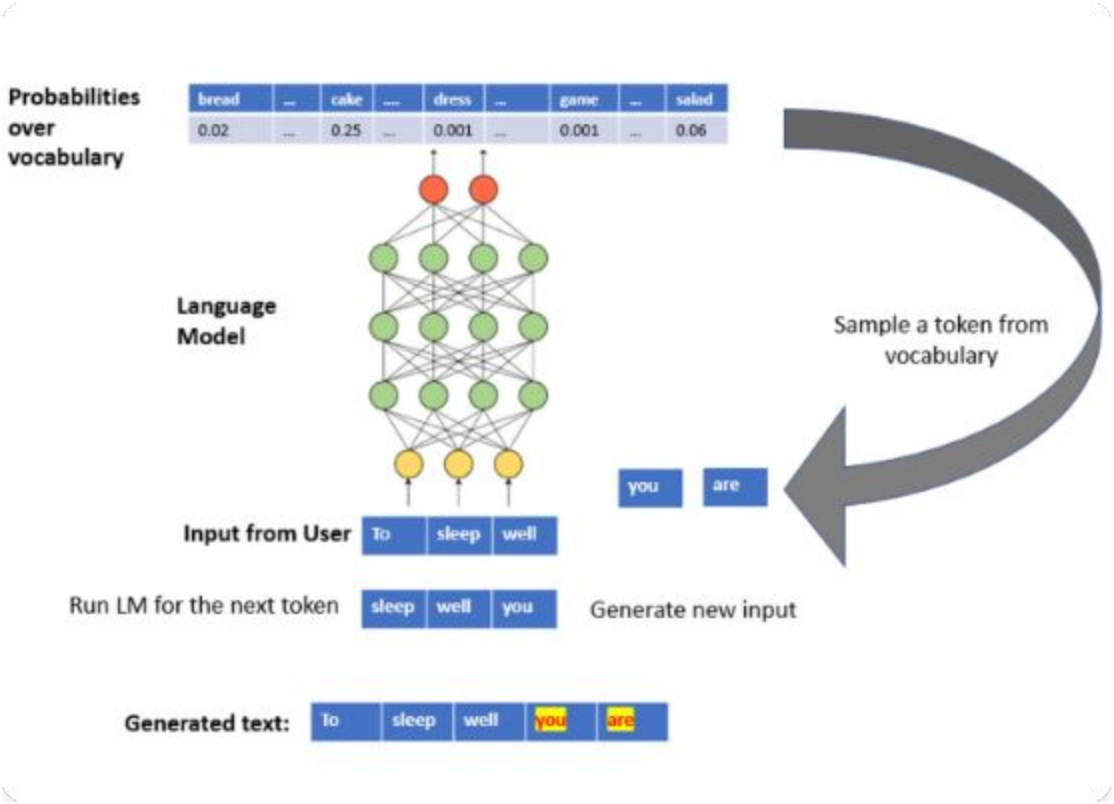
GRUs are similar to LSTMs but simplified. They combine the forget and input gates into a single "Update Gate".

### Advantages

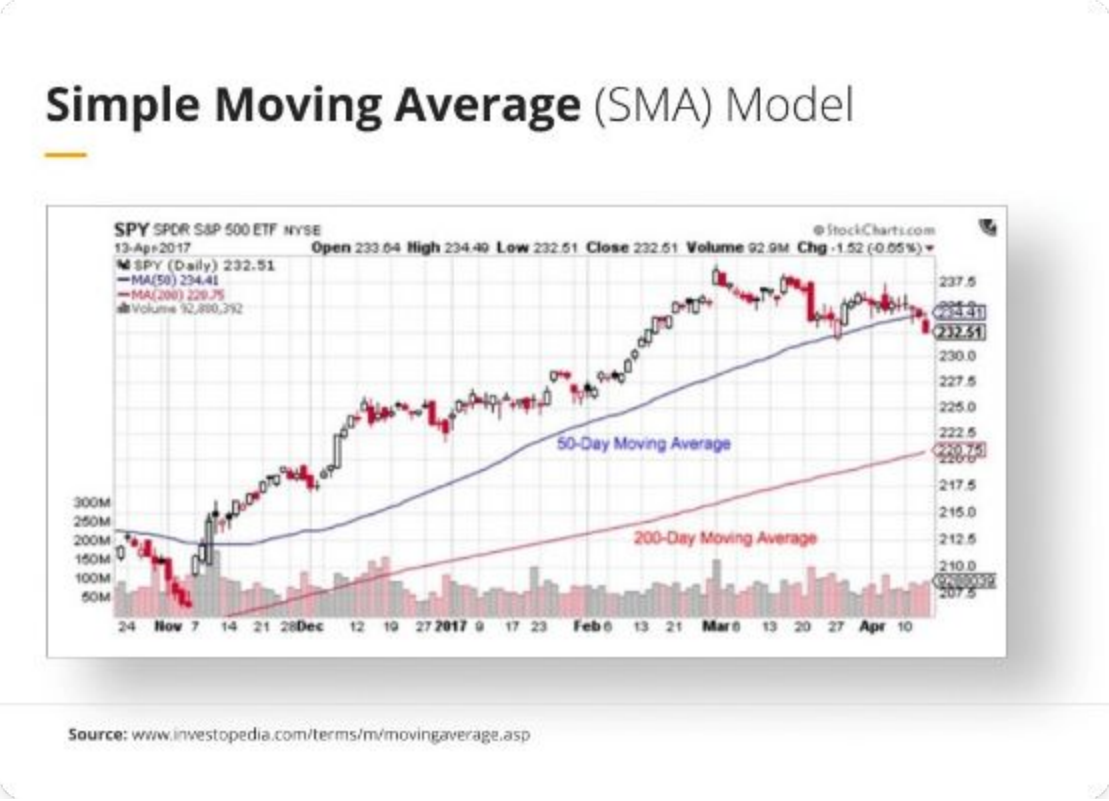
GRUs have fewer parameters than LSTMs, making them faster to train and less prone to overfitting on smaller datasets.

# Application: Natural Language Processing

- **Translation:** Google Translate (Seq2Seq).
- **Text Generation:** Chatbots, creative writing tools.
- **Sentiment Analysis:** Understanding customer feedback.
- **Speech Recognition:** Siri, Alexa (Audio waves to Text).



# Application: Time Series Forecasting




- **Stock Market:** Predicting future prices based on history.
- **Weather:** Forecasting temperature/rainfall.
- **Anomaly Detection:** Identifying unusual patterns in server logs or heart rate monitors.

# Summary & Key Takeaways

### Memory is Key

RNNs are powerful because they maintain state, allowing them to process sequences.

### Shared Weights

Using the same parameters across time steps makes them efficient but introduces gradient challenges.

### Modern Evolution

While basic RNNs are limited, LSTMs and GRUs solve the vanishing gradient problem, powering many modern AI systems.